

## Aim :- Association Rule Mining

1. Implement the Association Rule Mining algorithm (e.g., Apriori) to find frequent itemsets.
2. Generate association rules from the frequent itemsets.

## Description :-

The code loads the Groceries dataset and groups products into individual customer transactions. It uses the Apriori algorithm to identify frequent itemsets based on minimum support. Association rules are then generated using confidence and lift to find relationships between products. Finally, the code displays the top frequent itemsets and association rules using bar graphs.

| Step | Method / Function                 | Purpose                                                        |
|------|-----------------------------------|----------------------------------------------------------------|
| 1    | <code>pd.read_csv()</code>        | Loads the Groceries dataset.                                   |
| 2    | <code>groupby()</code>            | Groups records by Member number and Date to form transactions. |
| 3    | <code>apply(list)</code>          | Creates a list of products for each transaction.               |
| 4    | <code>TransactionEncoder()</code> | Creates a transaction encoder for market basket data.          |
| 5    | <code>fit_transform()</code>      | Converts transactions into a Boolean transaction matrix.       |

|    |                                                          |                                                        |
|----|----------------------------------------------------------|--------------------------------------------------------|
| 6  | <code>pd.DataFrame()</code>                              | Creates the transaction matrix.                        |
| 7  | <code>apriori()</code>                                   | Finds frequent itemsets using minimum support of 0.01. |
| 8  | <code>sort_values()</code>                               | Sorts frequent itemsets by support.                    |
| 9  | <code>association_rules()</code>                         | Generates association rules using confidence.          |
| 10 | <code>sort_values()</code>                               | Sorts rules by lift.                                   |
| 11 | <code>strong_rules</code>                                | Filters rules using confidence > 0.50 and lift > 1.    |
| 12 | <code>head()</code>                                      | Selects the top frequent itemsets/rules.               |
| 13 | <code>plt.barh()</code>                                  | Creates horizontal bar graphs.                         |
| 14 | <code>plt.xlabel()</code> /<br><code>plt.ylabel()</code> | Labels graph axes.                                     |
| 15 | <code>plt.title()</code>                                 | Adds graph titles.                                     |
| 16 | <code>plt.show()</code>                                  | Displays the graphs.                                   |

## CODE:

```

T090_PRACTICAL9_Alpy - C:\practical\AI\T090_PRACTICAL9_Alpy (3.13.0)
File Edit Format Run Options Window Help
import pandas as pd
import matplotlib.pyplot as plt

from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

file_path = "Groceries_dataset.csv"

df = pd.read_csv(file_path)

print("Dataset loaded successfully!")

print("\nFirst 5 rows:")
print(df.head())

print("\nDataset shape:")
print(df.shape)

print("\nColumn names:")
print(df.columns.tolist())

transactions = df.groupby(
    ["Member_number", "Date"]
)["itemDescription"].apply(list).tolist()

print("\nNumber of transactions:", len(transactions))

te = TransactionEncoder()

```

```

T090_PRACTICAL9_Alpy - C:\practical\AI\T090_PRACTICAL9_Alpy (3.13.0)
File Edit Format Run Options Window Help
te_array = te.fit(transactions).transform(transactions)

basket = pd.DataFrame(
    te_array,
    columns=te.columns_
)

print("\nTransaction Matrix:")
print(basket.head())

frequent_itemsets = apriori(
    basket,
    min_support=0.01,
    use_colnames=True
)

frequent_itemsets["itemsets_length"] = frequent_itemsets[
    "itemsets"
].apply(len)

print("\nFrequent Itemsets:")

print(
    frequent_itemsets
    .sort_values(
        by="support",
        ascending=False
    )
    .head(20)
)

```

```
T090_PRACTICAL9_Alpy - C:\practical\AI\T090_PRACTICAL9_Alpy (3.13.0)
File Edit Format Run Options Window Help

)

rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.30
)

rules = rules.sort_values(
    by="lift",
    ascending=False
)

print("\nAssociation Rules:")
print(
    rules.head(20).to_string(
        index=False
    )
)

strong_rules = rules[
    (rules["confidence"] > 0.50) &
    (rules["lift"] > 1)
]

print("\nStrong Association Rules:")

Ln: 166 Col: 19
23°C Partly cloudy 19:49 03-09-2026
```

```
T090_PRACTICAL9_Alpy - C:\practical\AI\T090_PRACTICAL9_Alpy (3.13.0)
File Edit Format Run Options Window Help

print(
    strong_rules
    .head(20)
    .to_string(index=False)
)

top_itemsets = frequent_itemsets.sort_values(
    by="support",
    ascending=False
).head(10)

itemset_labels = [
    ", ".join(list(itemset))
    for itemset in top_itemsets["itemsets"]
]

plt.figure(figsize=(10, 6))

plt.barh(
    itemset_labels[::-1],
    top_itemsets["support"].values[::-1]
)

plt.xlabel("Support")
plt.ylabel("Itemset")
plt.title("Top 10 Frequent Itemsets")

plt.tight_layout()

Ln: 166 Col: 19
23°C Partly cloudy 19:49 03-09-2026
```

```

plt.show()

top_rules = rules.head(10).copy()

rule_labels = [
    ", ".join(list(row["antecedents"])) +
    " → " +
    ", ".join(list(row["consequents"]))
    for _, row in top_rules.iterrows()
]

plt.figure(figsize=(10, 6))

plt.barh(
    rule_labels[::-1],
    top_rules["lift"].values[::-1]
)

plt.xlabel("Lift")
plt.ylabel("Association Rule")
plt.title("Top 10 Association Rules by Lift")

plt.tight_layout()

plt.show()

print("\n=====")
print("ASSOCIATION RULE MINING SUMMARY")

```

```

print("ASSOCIATION RULE MINING SUMMARY")
print("=====")

print("\nTotal transactions:", len(transactions))

print(
    "\nNumber of frequent itemsets:",
    len(frequent_itemsets)
)

print(
    "\nNumber of association rules:",
    len(rules)
)

print(
    "\nNumber of strong rules:",
    len(strong_rules)
)

print("\nTop 5 Rules:")

print(
    rules.head(5).to_string(
        index=False
    )
)

print("\n=====")

```

## OUTPUT:

```

Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/practical/AI/T090_PRACTICAL9_AI.py =====
Dataset loaded successfully!

First 5 rows:
  Member_number      Date  itemDescription
0          1808  21-07-2015    tropical fruit
1          2552  05-01-2015      whole milk
2          2300  19-09-2015        pip fruit
3          1187  12-12-2015    other vegetables
4          3037  01-02-2015      whole milk

Dataset shape:
(38765, 3)

Column names:
['Member_number', 'Date', 'itemDescription']

Number of transactions: 14963

Transaction Matrix:
  Instant food products  UHT-milk  ...  yogurt  zwieback
0                False      False  ...    True    False
1                False      False  ...   False    False
2                False      False  ...   False    False
3                False      False  ...   False    False
4                False      False  ...   False    False

[5 rows x 167 columns]

```

```

40  0.122101  frozenset({other vegetables})  1
46  0.110005  frozenset({rolls/buns})  1
52  0.097106  frozenset({soda})  1
63  0.085879  frozenset({yogurt})  1
47  0.069572  frozenset({root vegetables})  1
57  0.067767  frozenset({tropical fruit})  1
5  0.060683  frozenset({bottled water})  1
49  0.060349  frozenset({sausage})  1
15  0.053131  frozenset({citrus fruit})  1
41  0.051728  frozenset({pastry})  1
42  0.049054  frozenset({pip fruit})  1
50  0.047584  frozenset({shopping bags})  1
10  0.046916  frozenset({canned beer})  1
4  0.045312  frozenset({bottled beer})  1
59  0.043708  frozenset({whipped/sour cream})  1
37  0.038896  frozenset({newspapers})  1
21  0.037760  frozenset({frankfurter})  1
6  0.037626  frozenset({brown bread})  1
43  0.037091  frozenset({pork})  1

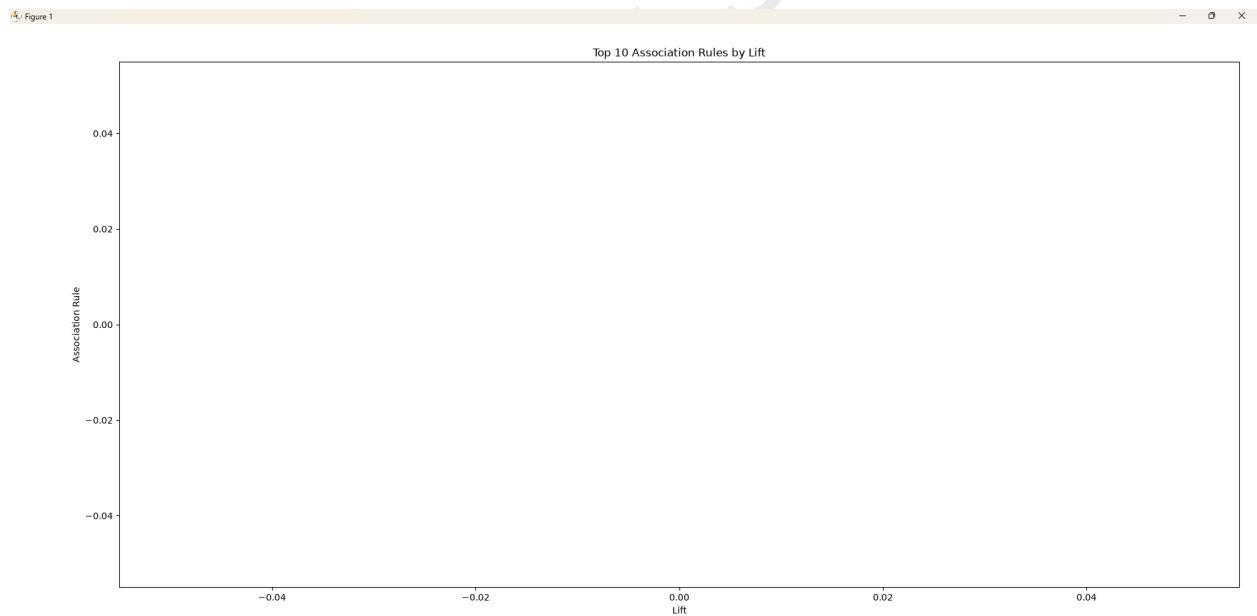
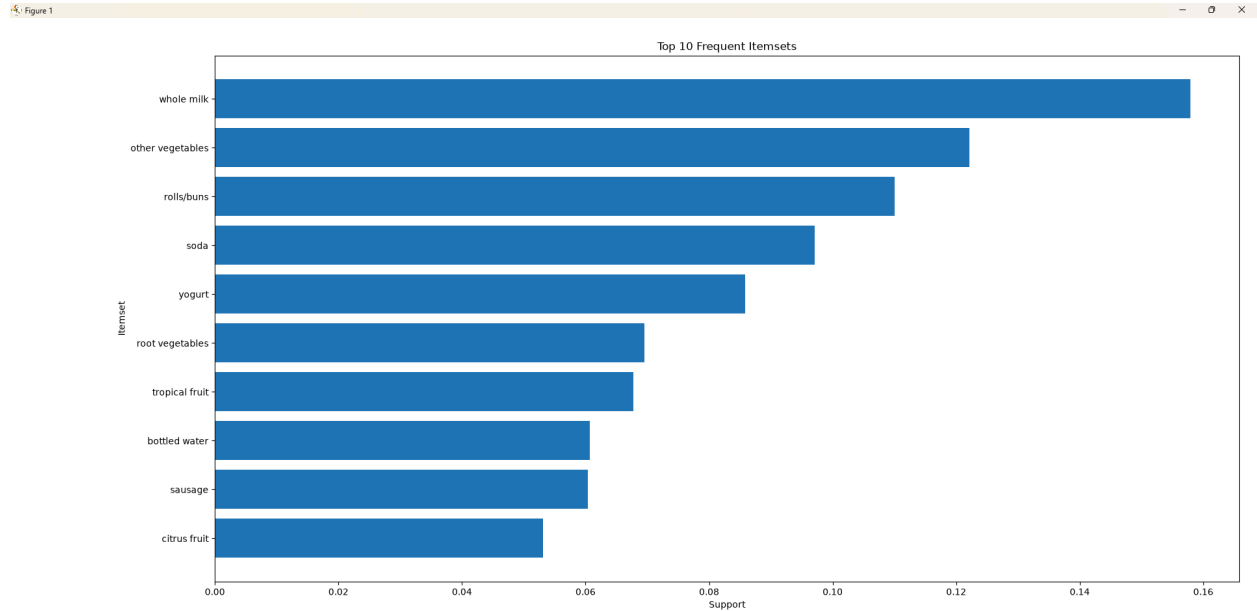
Association Rules:
Empty DataFrame
Columns: [antecedents, consequents, antecedent support, consequent support, support, confidence, lift, representat
ivity, leverage, conviction, zhangs_metric, jaccard, certainty, kulczynski]
Index: []

Strong Association Rules:
Empty DataFrame
Columns: [antecedents, consequents, antecedent support, consequent support, support, confidence, lift, representat
ivity, leverage, conviction, zhangs_metric, jaccard, certainty, kulczynski]
Index: []

```

# SHETH L.U.J. AND SIR M.V. COLLEGE

GRAPH:



## Observation

1. Top 10 Frequent Itemsets:

The graph shows that some grocery items or item combinations occur more frequently than others. Itemsets with higher support represent the most commonly purchased products.

2. Association Rules:

Association rules are generated using confidence and lift. For the selected minimum confidence of 0.30, no association rules were generated for this dataset.

3. The Apriori algorithm successfully identifies frequently occurring product combinations from the grocery transactions.

## Conclusion

Association Rule Mining was successfully implemented using the Apriori algorithm on the Groceries dataset. The transactions were created by grouping products according to Member-number and Date. Frequent itemsets were identified using a minimum support of 0.01. The practical successfully demonstrates how Apriori can be used to identify common purchasing patterns in grocery transaction data.